

Synthèse de révision du code

Ce document sert à comprendre et présenter le projet. L'objectif n'est pas de mémoriser chaque ligne, mais de savoir suivre une requête, expliquer une règle métier et retrouver le fichier responsable.

1. Le modèle mental

Une page suit presque toujours ce trajet :

```
URL -> route -> contrôleur -> modèle/service -> vue -> layout
```

Exemple pour `?page=menu-show&id=1` :

1. `public/index.php` trouve la route `menu-show`.
2. Il appelle `MenuController::show()`.
3. Le contrôleur valide `id`.
4. `Menu` récupère le menu et `Dish` sa composition.
5. Le contrôleur transmet ces données à `menus/show.php`.
6. `main.php` entoure la vue avec navigation et pied de page.

Si tu sais refaire ce chemin pour une fonctionnalité, tu comprends le MVC du projet.

2. Où chercher

Besoin	Fichier principal
Ajouter une route	<code>public/index.php</code>
Répondre à une requête	<code>app/Controllers/*Controller.php</code>
Lire ou écrire MySQL	<code>app/Models/*.php</code>
Lire ou écrire MongoDB	<code>Review.php</code> , <code>OrderAnalytics.php</code>
Calculer un prix	<code>app/Services/OrderPricing.php</code>
Envoyer un e-mail	<code>app/Services/MailService.php</code>

Besoin	Fichier principal
Modifier l'affichage	<code>app/Views/</code>
Modifier le visuel	<code>public/css/style.css</code>
Modifier une interaction	<code>public/js/app.js</code>
Modifier les filtres	<code>public/js/menu-filters.js</code>
Modifier les tables	<code>database/schema.sql</code> et <code>database/migrations/</code>

3. Ce qu'est réellement le MVC

- **Modèle** : connaît la base et les transactions, pas le HTML.
- **Vue** : affiche les variables, sans requête SQL.
- **Contrôleur** : valide la requête, vérifie les droits, appelle les modèles et choisit la vue ou la redirection.
- **Service** : contient une logique réutilisable qui n'appartient ni à la base ni à l'affichage.

Exemple : le calcul tarifaire est un service parce qu'il ne dépend ni de PDO ni du navigateur.

4. Authentification

Inscription

`AuthController::register()` :

1. refuse un utilisateur déjà connecté ;
2. récupère et nettoie les valeurs POST ;
3. transmet les coordonnées, le mot de passe et les consentements à `UserRegistrationService` ;
4. le service valide les coordonnées obligatoires et l'unicité de l'e-mail ;
5. il appelle `User::createCustomer()` avec un mot de passe haché ;
6. il prépare l'e-mail de bienvenue ;
7. le contrôleur affiche la confirmation.

Le rôle créé par l'inscription est toujours `user`. `PasswordPolicy` applique partout la même règle : 10 à 72 caractères avec une majuscule, une minuscule, un chiffre et un caractère spécial.

Connexion

`AuthController::login()` :

1. limite les échecs répétés ;
2. cherche un compte actif par e-mail ;
3. utilise `password_verify()` ;
4. régénère l'identifiant de session ;
5. stocke uniquement les informations utiles dans `$_SESSION['user']` ;
6. redirige selon le rôle.

Redirections :

- client vers `menus` ou la commande qu'il voulait commencer ;
- employé vers `employee-orders` ;
- administrateur vers `admin-dashboard` .

Mot de passe oublié

Le jeton brut part dans l'URL. La base ne stocke que son SHA-256. Il expire après une heure et `used_at` empêche une seconde utilisation.

5. Autorisations

Les contrôles importants sont dans `App\Core\Controller` :

```
$this->requireUser();  
$this->requireRole(['employee', 'admin']);  
$this->requireRole(['admin']);
```

Masquer un lien n'est pas une sécurité. Même si un client écrit manuellement `?page=admin-users`, le contrôleur doit refuser.

Pour une commande client, le modèle cherche avec l'identifiant de commande **et** l'identifiant du client. C'est la protection contre l'accès à la commande d'un autre utilisateur.

6. Menus et plats

Un menu contient ses données dans `menus`, plusieurs images dans `menu_images` et plusieurs plats via la table de liaison `menu_dishes` .

La relation menu/plat est **N-N** :

- un menu possède plusieurs plats ;
- un plat peut être utilisé dans plusieurs menus.

`Menu::create()` et `Menu::update()` utilisent une transaction pour enregistrer le menu, sa composition et sa galerie ensemble. En cas d'erreur, tout est annulé.

Un menu doit contenir au moins :

- une entrée ;
- un plat ;
- un dessert.

Un plat utilisé par un menu ne peut pas être supprimé. Un menu est archivé, pas supprimé : stock à zéro et visibilité inactive.

7. Calcul d'une commande

Exemple : menu à 120 € pour 4 personnes, commandé pour 9 personnes à 8 km de Bordeaux.

```
prix menu = 120 × (9 / 4) = 270 €  
remise = 270 × 10 % = 27 €  
livraison = 5 + (8 × 0,59) = 9,72 €  
total = 270 - 27 + 9,72 = 252,72 €
```

Pourquoi calculer en JavaScript **et** en PHP ?

- JavaScript améliore l'expérience en affichant le prix immédiatement.
- PHP protège l'intégrité car le client peut modifier le JavaScript.

Le montant enregistré vient toujours de `OrderPricing` côté serveur.

8. Transaction et stock

`Order::create()` est le point le plus important à comprendre :

1. `beginTransaction()` ;
2. sélection du menu avec `FOR UPDATE` ;
3. contrôle du stock ;
4. insertion de la commande ;

5. décrémentation du stock ;
6. insertion du statut initial ;
7. `commit()` .

`FOR UPDATE` empêche deux requêtes simultanées de commander le dernier exemplaire. Si une étape échoue, `rollback()` laisse la base cohérente.

Lors d'une annulation, le stock est restauré dans la même transaction.

9. Cycle des statuts

```
nouvelle
-> accepte
-> en_preparation
-> en_livraison
-> livre
-> attente_materiel
-> terminee
```

Une annulation est autorisée depuis les statuts opérationnels. Le contrôleur détermine les transitions permises ; le modèle enregistre le changement et son historique.

Pourquoi une table d'historique plutôt qu'un seul champ ?

Le champ `orders.status` donne l'état actuel. `order_status_history` prouve qui a fait chaque changement, quand et avec quel commentaire.

10. Annulation

Client :

- uniquement avant acceptation ;
- motif automatique ;
- stock restauré.

Employé :

- contact préalable obligatoire ;
- moyen `telephone` ou `email` ;
- motif de 10 à 500 caractères ;
- notification et e-mail au client ;

- stock restauré.

11. MongoDB

Les avis sont des documents car leur structure est autonome et se prête au stockage NoSQL. Ils passent par `pending`, `validated` ou `refused`.

Les statistiques utilisent une seconde collection, `order_analytics`, qui est une projection des commandes SQL. MongoDB réalise ensuite les regroupements par menu.

Pourquoi garder MySQL pour les commandes ?

Les commandes, utilisateurs, menus et stocks ont de fortes relations et nécessitent des transactions. Une base relationnelle est adaptée.

Pourquoi un secours SQL ?

MongoDB est externe. Une panne ne doit pas bloquer le tableau de bord ni l'accueil. Le code affiche clairement la source utilisée.

12. Sécurité à savoir expliquer

Mécanisme	Risque réduit
requête préparée PDO	injection SQL
<code>htmlspecialchars</code>	injection HTML/JavaScript
jeton CSRF	action envoyée depuis un autre site
<code>password_hash</code>	exposition d'un mot de passe en base
régénération de session	fixation de session
contrôle de rôle	accès interdit
jeton reset haché et expirant	vol ou réutilisation
limitation de connexion	brute force
configuration ignorée par Git	fuite de secrets
HTTPS + cookie Secure	interception de session

Question classique : « Pourquoi valider côté serveur si le champ HTML est `required` ? » Parce qu'une requête HTTP peut être envoyée sans utiliser le formulaire.

13. CSRF

`csrf_field()` ajoute un champ caché lié à la session. Au POST, `csrf_is_valid()` compare le jeton avec `hash_equals`.

À utiliser pour toute action qui change un état :

- création ou modification ;
- annulation ;
- changement de statut ;
- déconnexion ;
- modération.

Un formulaire de recherche en GET n'a pas besoin de CSRF puisqu'il ne modifie aucune donnée.

14. E-mails et notifications

`MailService` prépare les messages pour :

- bienvenue après inscription ;
- confirmation de commande ;
- changement de statut ;
- compte employé ;
- mot de passe oublié ;
- formulaire de contact.

Les notifications SQL alimentent le badge client. L'e-mail informe hors du site, la notification reste consultable après la connexion.

Un échec d'e-mail est journalisé mais ne doit pas annuler une commande déjà validée en base.

15. JavaScript

`menu-filters.js` envoie les critères à `?page=api-menus` avec `fetch`. Le serveur renvoie les identifiants autorisés et le script met à jour les cartes sans recharger la page.

`app.js` :

- masque les messages de confirmation après un délai ;
- demande confirmation avant une action sensible ;
- active/désactive les champs d'horaires ;

- affiche les champs d'annulation employé ;
- met à jour le résumé tarifaire.

Le site reste utilisable pour les actions principales sans dépendre d'une API JavaScript.

16. Résilience

Si MongoDB tombe :

- l'accueil s'affiche sans avis ;
- le compte client garde ses commandes ;
- les pages d'avis renvoient une erreur 503 compréhensible ;
- les statistiques basculent sur MySQL.

Si une exception inattendue survient :

- en développement, le message aide au débogage ;
- en production, le visiteur voit un message neutre ;
- le détail est écrit dans les logs.

17. Tests

```
composer test
```

Teste l'architecture et les règles pures sans base.

```
composer test:integration
```

Crée des données temporaires, vérifie relations, galerie, prix, stock, historique et annulation, puis nettoie la base.

Savoir expliquer la différence :

- test unitaire/domaine : rapide et isolé ;
- test d'intégration : vérifie plusieurs composants et une vraie base ;
- recette : vérifie le comportement visible dans le navigateur.

18. Questions probables du jury

Pourquoi MVC ?

Pour séparer les responsabilités, faciliter les tests et éviter un fichier `index.php` contenant SQL, validation et HTML.

Pourquoi PDO ?

PDO fournit une interface cohérente, les requêtes préparées et les transactions.

Pourquoi deux bases ?

MySQL garantit relations et transactions métier. MongoDB stocke les documents d'avis et exécute les agrégations NoSQL demandées.

Comment empêches-tu un prix falsifié ?

Le navigateur affiche une estimation, mais PHP recalcule le prix depuis le menu lu en base avant l'insertion.

Comment empêches-tu une rupture de stock ?

Transaction, verrou `FOR UPDATE`, contrôle puis décrémentation conditionnel.

Pourquoi archiver un menu ?

Les commandes anciennes doivent continuer à référencer ce menu. Le supprimer casserait l'historique.

Que se passe-t-il si MongoDB est indisponible ?

Le code capture l'erreur. Les fonctions SQL restent actives et les statistiques utilisent un calcul de secours.

Qu'est-ce qui reste à faire hors code ?

Déployer, tester les e-mails réels, renseigner l'URL, le tableau projet et l'identité légale définitive, puis effectuer la recette de production.

19. Exercices pour t'appropriier le code

1. Suivre sur papier la route `employee-menu-edit`.
2. Expliquer chaque requête de la transaction `Order::create()`.
3. Calculer trois commandes à la main puis comparer à `OrderPricing`.
4. Trouver tous les `requireRole` et dire quel rôle est attendu.
5. Ajouter mentalement un statut et lister tous les fichiers à modifier.
6. Désactiver MongoDB localement et observer les comportements de secours.
7. Présenter le MCD sans lire la documentation.

Quand ces sept exercices sont clairs, tu peux défendre l'essentiel du projet sans réciter le code ligne par ligne.